

Table of Contents

System Architecture.....	3
Main Components.....	3
How They Communicate.....	3
Alternative routes logic.....	4
Client server communication.....	4
Traffic updates.....	5
Accuracy-Runtime Tradeoff.....	5
API & UML.....	6
Algorithms and Data Structures.....	7
The Live Networks Problem.....	7
The CRP Solution.....	7
The Overlay Graph.....	7
Graph Partitioning Algorithm.....	8
Routing Algorithm.....	8
The Customization Phase.....	9
Data Structures.....	10
Runtime Objects and Entities.....	12
Special Features & Optimizations.....	12
Parallel and Serial Architecture.....	13
Embarrassingly Parallel Actions:.....	13
Serial Parts:.....	14
Concurrency Tools.....	14
Benchmarks.....	15
CRP vs A*	15
Server Scalability.....	16

System Architecture

The project's architecture relies on a highly concurrent Go server designed to handle a great number of users continuously updating their locations and requesting navigation routes.

Main Components

The system acts as a central server running many independent goroutines that continuously interact with shared, thread-safe data access layers.

1. API Server: Handles incoming user requests for routing and session data.
2. Session Manager: Monitors active user driving sessions, holds Estimated Time of Arrival (ETA) metrics, and flags users when traffic heavily alters their route times or when they go off route, which leads to a reroute calculation.
3. Traffic Store: The central traffic manager. It collects raw speed samples from all live drivers, applies decay logic, and generates weighted costs for each edge (road).
4. Router: Wraps the Base and Overlay Graphs and provides shortest paths using Customizable Route Planning (CRP) algorithm.
5. Simulation Manager: Simulates many driving cars, producing realistic speed samples.
6. Client Application: An website where users can search locations, visualize optimal routes, and navigate with real-time traffic updates.
7. Graph builder: A preprocessing tool engine to parse OpenStreetMap PBF data of any part in the world into a two-layered graph, using Inertial Flow Recursive Bisection algorithm to separate into cells and serializing the output for fast reload.

How They Communicate

Services maintain constant communication via Go Channels and synchronize using RWMutexes.

- Rest API: Geolocation and routing requests are caught by the API manager and redirected to the search handler or routing engine.
- Client pings: As a client navigates, their device broadcasts high-frequency WebSocket "pings", containing information about their current location and speed. The API funnels this telemetry into the Traffic Store.
- Traffic Aggregation: The Traffic Store processes this incoming data asynchronously. Background Traffic Sweepers continuously apply Exponentially Weighted Moving Average (EWMA) and density counters to calculate "live weights" (time delays) for edges, and decay weights back to their base when roads empty out. Simultaneously, a routine picks up these weight updates in the base graph and computes updated weights to overlay edges.
- Session Evaluation: The Session Manager's Propagation worker runs in the background, observing weight changes from the Traffic Store. If it spots major ETA variances via our

Admissible Heuristics Filters - it flags the session as “potentially has a shorter route now”.

- Rerouting calculations: Finally, the API's RunOptimizationSweep worker catches those flagged sessions and dispatches real-time recalculation requests to the Router. It finds a better route for the user either with localized repairs (Path Patching) or full graph recounts, sending the new route back to the client.
- Simulation: With the simulation we can test the behaviour and performance of the system when many clients are connected simultaneously, by injecting thousands of mock-drivers into the same API pipeline as real clients, generating traffic jams artificially. To make the simulation realistic, car speed is adjusted based on the amount of cars on the same edge.

Alternative routes logic

To ensure the best user experience and maintain optimal travel times, we dynamically reroute the user in the following three scenarios:

- Traffic detected: On every client ping, the server compares recent traffic updates against the user's current route, updating ETA and also calculating an alternative route if a significant congestion is detected ahead. Sometimes, the traffic is converged into one specific road or part of a road, so the server checks the possibility for surrounding the local traffic using Local Patching optimization (explained later in the special optimizations passage) instead of calculating the entire route from the beginning.
- Traffic cleared: When the propagation worker detects a significant weight improvement, it uses a fast heuristic to flag vehicles who might benefit from the cleared road (cars whose theoretical optimal distance to the edge endpoint + edge weight is less than their ETA). The RunOptimizationSweep worker then performs a full route calculation on flagged cars to verify the intuition before rerouting them.
- Off-route: If the user is not driving on the route it was given, the ping handler will notice and calculate a fresh new route from the client's new location.

Client server communication

The communication between the client and the server contains the following steps:

- Location search: The user enters source and destination addresses. The server geocodes (translates textual address to coordinates) them using [Nominatim](#).
- Route search: The user clicks “navigate”, sending a /route request to the server. The server runs the routing algorithm and returns the optimal route, and to a few alternatives. The user can view the routes on the map and compare their stats.
- Start driving: The user clicks “Start driving, and a POST /session request is sent to the server with the chosen route id (the server remembers the route for up to 30m).

- Driving: The car moves in the simulation, mimicking real car physics (random accelerations, slowing down at turns, stopping behind other cars etc). The users constantly update the server about their speed, route progress, and edge travel time. The server constantly updates ETA updates and reroutes when needed.

We also added a simulation system that creates many cars with random routes and displays them on the map. The cars are handled on the server and sent to the client side to display on the map.

Traffic updates

The client sends two types of speed updates to the server:

- Speed samples the user sends every few ticks. They give us live and frequent updates, but have a lot of noise and are affected by natural slow-downs and speed-ups.
- The time it took the user to cross this edge. This is a much more stable measure of the congestion, but the update is sent from the user only when it exits the edge.

The weight is updated using an exponential weighted moving average (EWMA):

$$W_{new} = \alpha \cdot \frac{observed}{base} + (1 - \alpha) \cdot W_{old}$$

The α value controls how much the new data affects the recorded average. It is higher for edge travel times since they are more reliable and lower for speed samples which include a lot of noise.

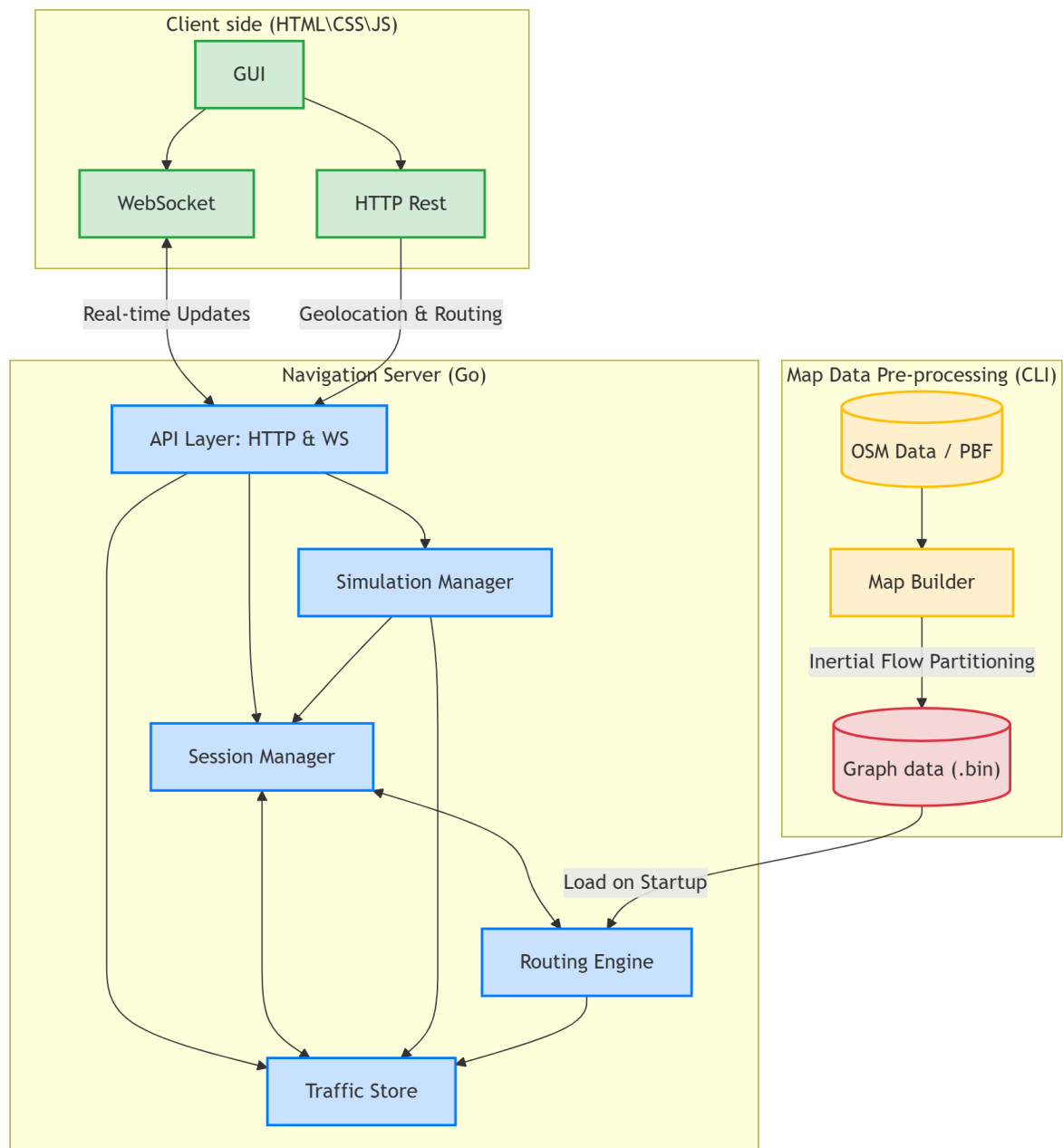
Accuracy-Runtime Tradeoff

The standard formula for calculating distance between points of Earth - the Haversine formula - is computationally expensive (it includes many cos, sin and arctan computations). To optimize, we can approximate the distance by projecting the nodes on a flat plain and calculating their Euclidean distance. The method to choose depends on the context:

- Road lengths are calculated using the accurate Haversine formula (maximum accuracy, negligible preprocessing overhead).
- Off Route violation, snapping and route progress are computed using standard Euclidean distance on projected nodes (full accuracy is not required).
- A* heuristic uses Euclidean distance on projected nodes + one cosine calculation per query (the sweet spot between speed and precision).

API & UML

URL	Method	User sends	Server sends
/search	GET	Location name	List of places (Name, lat, lng)
/route	POST	Src-Dst coords, alternatives count	List of Route objects
/session	POST	Route id (from /route call)	Session id
/session/{id}/ws	WS	Speed, route progress, past edges travel times	ETA updates, reroutes
/simulation	POST	Route id, car count to spawn	Created cars count
/simulation/random	POST	Car count to spawn	Created cars count
/simulation/ws	WS	(None)	Simulated cars status



Algorithms and Data Structures

In this project we implemented the Customizable Route Planning (CRP)¹ algorithm. The algorithm was developed by Daniel Delling, Andrew V. Goldberg, Thomas Pajor, and Renato F. Werneck at Microsoft Research, to enable real-time route calculation on large and dynamic road networks.

The Live Networks Problem

Modern navigation systems like Waze must handle massive amounts of users at the same time, making standard algorithms like Dijkstra's or A* way too slow, which is why modern techniques rely on map preprocessing to enable quick queries at runtime. However, these precomputed data structures often take hours to generate, while live traffic updates every second. The system then fails to adapt quickly to real-time traffic jams, road blocks, or personalized routing preferences.

The CRP Solution

Customizable Route Planning separates the map structure (topology) and the dynamic costs (metric). First, a one-time preprocessing phase partitions the graph and constructs a small overlay. Then, an embarrassingly parallel "customization" phase injects live traffic into edge weights every few seconds. Because the topology and metric are independent, it only updates edge weights, skipping the expensive graph rebuild. Queries use the preprocessed overlay graph to skip massive parts of the map, executing instantaneously.

The Overlay Graph

The main idea behind the overlay graph is to divide the map into a collection of distinct neighborhoods, called *cells*. By identifying which roads enter and exit the cells, the preprocessor connects *shortcut edges* from cell entrances to its exits. At runtime, the routing algorithm uses the precalculated shortcut weights to jump straight to the exits and skip the entire neighborhood.

To achieve this the system keeps two main data structures:

1. The Base Graph: The original full graph representing the map. In this graph, nodes represent physical locations and edges represent road segments.
2. The Overlay Graph: A skeleton of the original graph, consisting of these three components:
 - *Gate Nodes (Boundary Nodes)*: Nodes with at least one edge going outside their cell.

¹ Daniel Delling, Andrew V. Goldberg, Thomas Pajor, Renato F. Werneck, "Customizable Route Planning in Road Networks":

- https://www.microsoft.com/en-us/research/wp-content/uploads/2013/01/crp_web_130724.pdf
- <https://renatowerneck.wordpress.com/wp-content/uploads/2016/06/dgpw11-crp-sea.pdf>

- *Cross Cell edges*: Road segments connecting gates of two different cells.
- *Shortcut edges*: Logical edges connecting every gate node to every gate node within the same cell. This creates a clique inside each cell, where the weight of the shortcut from a to b is set to the weight of the shortest route from a to b inside the cell.

The overlay graph omits all inter cell nodes and edges, making it much smaller.

Graph Partitioning Algorithm

The algorithm used to partition the graph into cells is called Inertial Flow Recursive Bisection², developed by Aaron Schild and Christian Sommer. Its primary purpose is to divide the graph into roughly equal sized cells, minimizing the number of cross edges connecting them. Unlike traditional clustering algorithms like K-Means, which groups nodes simply by their Euclidean proximity, Inertial flow accurately measures real world geography (like natural rivers/bridges). In the first step of the algorithm, we bisect the graph into two halves with a random-angled line, and project nodes onto it. We group nodes that fall in the two extremes, set them as super-source and super-sink, and run Dinic's Min-cut\Max-flow algorithm (BFS level-graphs + DFS flow augmentations) to partition the graph by the fewest amount of crossing edges. We recursively divide the resulting subgraphs in parallel until the cells match our desired dimensions, resulting in a small sized overlay graph.

Routing Algorithm

Finding the shortest route between two points involves a Two-Level Search on the base and overlay graphs. Given source (s) and target (t) coordinates, we perform the following steps:

1. Find the closest node to s (sign it as s') and the closest node to t (sign it as t').
2. Identify the cell containing s' (Cell A) and the cell containing t' (Cell B).
3. Run Dijkstra inside cell A to find the shortest distance from s' to every gate of cell A.
4. Once on the gates of Cell A, hop onto the overlay graph. Using the distances from the first Dijkstra as starting costs, execute A* with the gates of cell B as destinations.
5. Once on the bounds of Cell B, hop back into the base graph and run Dijkstra inside cell B to find the shortest path from the gates to t'.

After those steps, we obtain a route constructed from base and cross-cell edges (that exist in the base graph) and "shortcut" logical edges (that are not in the base graph). Those edges are then translated into the shortest routes they represent. The naive approach is to store the full path for every shortcut edge when it's created. This approach is very memory wasteful, so the CRP approach is to run a quick Dijkstra inside the edge's cell to retrieve the path. Since the cells are small, it barely affects runtime (a cell Dijkstra takes a few milliseconds). Connecting all of these subroutes together, we obtain the complete shortest route from s to t in the original graph!

² Aaron Schild and Christian Sommer, "On Balanced Separators in Road Networks":

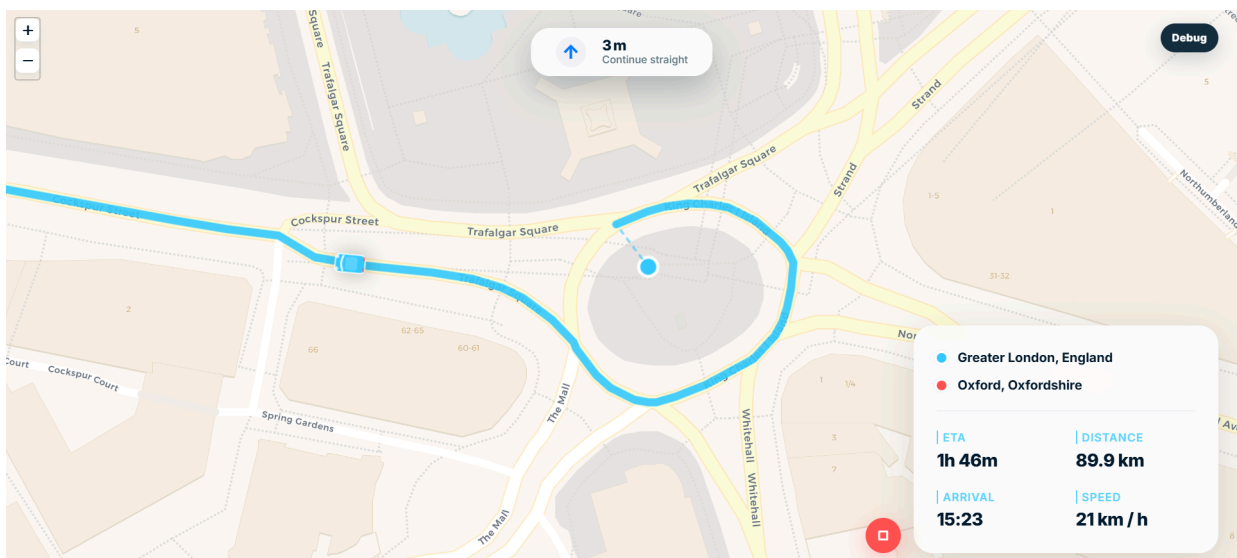
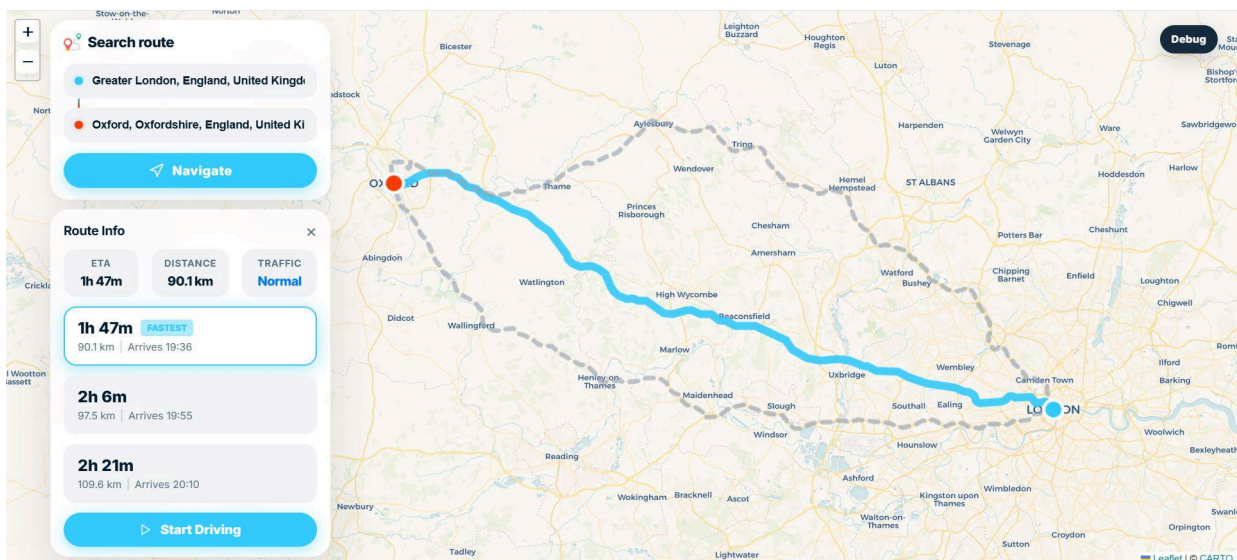
- <https://aschild.github.io/papers/roadseparator.pdf>

The Customization Phase

When traffic changes, shortest paths change as well, making existing shortcut edges outdated. To address this, we periodically run a *customization* step in which we recompute all shortcut weights. For each gate node, we run Dijkstra inside its cell to get the distances to every other gate in that cell. We use those distances to update the weights of the corresponding shortcut edges.

Because each gate's computation is independent, they can all be computed in parallel. In practice, the number of threads is limited and we need to partition the work. We grouped the gates by cell, assigned each cell to a task and computed all tasks concurrently.

At the start of a phase, we check which cells are affected by the traffic changes and recompute their shortcut weights. We compute the weights for all the shortcuts in the cell and not just the ones that are affected. This is because the cell Dijkstra is so fast, that calculating all the weights is faster than filtering the edges that have been affected by the changes.



Data Structures

Graph structure

The following section details the data structures used in our implementation of the CRP algorithm. The map data is real world data obtained from OpenStreetMap (OSM) using Geofabrik.

The basic entities forming the graph are nodes, edges, overlay edges (shortcut or cross-cell) and cells. A cell stores its gate nodes as a list of their indices in the Nodes array.

The data structure was designed according to the **spatial locality principle**. Nodes and edges in Nodes[], Edges[] and OverlayEdges[] are grouped by cells, so elements that are close in the real world are also close in memory. When one item is accessed, nearby elements are brought into the cache as well. Since routing algorithms are built on the idea of exploring nearby nodes and edges, this design improves **cache utilization**.

The base and overlay graphs are represented in **Compressed Sparse Row (CSR)** this format is composed of two arrays:

- Edges array: A single array containing all edges (or edge IDs) sorted by their source node.
- Offsets array: Offsets[i] points to the index in Edges where Node i's outgoing edges begin.

We also store geographical metadata about the graph:

- Bounding box: Contains min lat, max lat, min lon, max lon. They form the geographical bounds on the map on Earth.
- RefLat: Computing accurate distances using Haversine formula is expensive, so the nodes are projected onto a flat (x,y) plain and use Euclidean distance instead. The RefLat reduces latitude projection distortion, keeping distances accurate.

Traffic Store

All live traffic data is stored in arrays in the *traffic store*. The base store stores the weight and the density (amount of cars on it) of each edge of the base graph. The shortcut store stores the weights of all shortcut edges. Storing all weights in a contiguous array allows us to pull cache lines full of weights only without additional data we don't need.

```
type Graph struct {
    Nodes      []Node
    Cells      []Cell
    Gates      []int32
    Base       BaseGraph
    Overlay     OverlayGraph
    BBox       BoundingBox
    RefLat     float64
}

type BaseGraph struct {
    Offsets    []uint32
    Edges      []Edge
}

type OverlayGraph struct {
    Offsets    []uint32
    Edges      []OverlayEdge
}

type Node struct {
    Lat, Lon   float64
    X, Y       float32a2
    CellID     uint32
}

type Edge struct {
    ID          uint32
    SrcNode, DstNode uint32
    BaseWeight   float32
    Length       float32
    SpeedLimit   float32
    Flags        uint8
}

type OverlayEdge struct {
    SrcNode, DstNode uint32
    BaseWeight       float32
    StoreIdx         uint32
    IsCrossCell      bool
}

type Cell struct {
    Gates []uint32
}
```

Similarly to the list of edges and nodes, those arrays are also sorted by cells. The field StoreIdx in the OverlayEdge struct points to the position in the lists where the weight of the edge is located (in base store if it is a cross cell, in shortcut store if it is a shortcut). All values are atomic integers, so concurrent readers can read and write them without races.

```
type baseStore struct {  
    weight    []atomic.Uint32  
    density   []atomic.Int32  
}  
  
type shortcutStore struct {  
    weight []atomic.Uint32  
}
```

In addition to the weights the traffic store also holds information about previous weights (to track changes over time and detect congestions), edges that no longer have balanced weight (so the decay worker will iterate only relevant edges rather than the entire map), edges that their weights had changed since the last customization (so we skip shortcut recalculations for cells that had no changes). The access to this structure shortens the worker runtime and prevents unnecessary work.

All those lists are guarded by a mutex that prevents concurrent workers from adding\removing edges from the same list at the same time, and from different lists at the same time (to ensure consistency between the lists, so we don't have contradictions about an edge's status).

```
type Store struct {  
    base      atomic.Pointer[baseStore]  
    shortcut  atomic.Pointer[shortcutStore]  
  
    prevWeight []float32    // Weight and density in previous snapshot  
    prevDensity []int32  
  
    dirtyEdges []model.EdgeID // Edges with multiplier ≠ 1.0  
    isDirty    []bool  
  
    pendingEdges []model.EdgeID // Edges with recent weight change  
    isPending    []bool  
  
    activityEdges []model.EdgeID // Edges with recent density change  
    isActive     []bool  
  
    metaMu    sync.Mutex // Protects concurrent list edits  
}
```

The live travel time is calculated by $\text{Travel time} = \text{Base weight} * \text{Multiplier}$. The base weight is the default driving time over an edge ($\text{length} / \text{speed limit}$), and the Multiplier is the value stored in the weight list in the traffic store. The multiplier represents how much an edge is congested, where a multiplier of 1.0 means no congestions (free edge), so the decay worker can skip them.

Runtime Objects and Entities

Beyond the map data, the server relies on volatile data structures mapping active states in memory to handle sessions, cars, requests, and routes:

- Session: Represents the mutable state of a single driver navigating. It encapsulates their assigned Route, expected ETA, recent GPS coordinates, congestion markers, and dynamic flags like OffRouteViolations. It securely manages connection pipelines utilizing a dedicated WebSocket SendChan queueing outgoing real-time communication messages.
- Route: A route is represented as a list of steps. A step is an edge with some extra information about its nodes. During route generation we extract all the relevant information node and edge data from the graph and send it to the user as a single route object.
- Car: The car object projects simulated cars during ticks. It contains progress meters, current simulated geographic speeds, base route steps and the session they are connected to.
- OutMsg: Standardized communication structs converting internal Session metrics (new ETA, off-route warnings, new route object on reroute) into JSON payloads pushed out to clients.

Special Features & Optimizations

Alternative Routing Choices: To provide multiple variant routing options to the user, we run the routing algorithm multiple times, penalizing previous routes. Subsequent runs dynamically inflate traversed overlay weights so alternate routes will naturally pick distinct paths.

Admissible Heuristics: To assist the A* approach natively on the Overlay Graph - and significantly slice response speeds across broad spans - the algorithm references an Admissible Heuristic equation calculating approximated Euclidean distance $\div$ maxSearchSpeedMps. A safe lower-bound heuristic guides the algorithm towards the target without over-analyzing trailing edge trees.

Local Route Patching: Recalculating the entire path every time a road gets congested can be computationally expensive. To optimize this, we implemented Local Route Patching. When a crossover edge (Edge A) experiences a large increase in traffic, the system halts the global recalculation and instead runs a bounded "mini-Dijkstra" between the two boundary nodes defining Edge A, seeking a fast local detour that surround Edge A (like 2 Edges B and C where $A=(x,y)$, $B=(x,w)$, $C=(w,y)$). The system evaluates the time delta - if the newly patched local detour resolves the traffic spike within an acceptable threshold, the system patches the original route without ever triggering an expensive computation on the global Overlay Graph.

Node snapping: When the user requests a route between two coordinates we first need to map them to real graph nodes, by finding the closest nodes to them (snapping). To avoid searching millions of nodes every time we build a spatial grid of buckets of nodes. To snap a node we can run the search only on the nodes in this small local bucket (and search other buckets only when their boundary is close enough to potentially contain a closer node).

Parallel and Serial Architecture

To fulfill processing demands, Go's native goroutines dictate workload concurrency.

Embarrassingly Parallel Actions:

- Graph Partitioning: During Inertial Flow Bisection, the initial map area recursively cuts itself in half. Each newly bisected child chunk directly maps to a distinct goroutine (via `spawn()` channels limiting bounded CPU resources) ensuring simultaneous calculation of Dinic's max-flow logic across diverging partitions.
- Overlay Graph Cells Building (Internal Cliques Calculation): In the building of the overlay graph cells, every logical shortcut edge inside every cell is calculated in parallel across cells, independently. Through dedicated worker channels, independent cells are distributed to a CPU worker pool to concurrently perform Dijkstra from gates to gates.
- Graph loading and serialization: All graph data structures are saved into bin files and loaded from them simultaneously.
- Overlay Customization Weights: Handling cell-level gate-to-gate Dijkstra shortest paths is divided amongst CPU worker groups natively to update dynamic traffic contexts rapidly.
- Microservice Pipelines: All systems like Traffic worker receivers, Propagation sweepers (`RunPropagation`), Optimization pipelines (`RunOptimizationSweep`), and concurrent tick simulation managers run asynchronously.
- Simulation Vehicle Generation: Generating random routes for the simulation is done concurrently using a worker pool, where each route is a task executed by a goroutine.
- Simulation Tick Update: Every tick the server advanced all the cars in the simulation simultaneously (the cars are split to chunks, each chunk taken by a goroutine).
- Local Route Repair (Path Patching) Sweeps: The optimization concurrently processes subsets of driver sessions detecting congestion hurdles applying parallel local bounded Overlay patch search algorithms averting global recalculations.
- Client handling: Different clients accessing the server at the same time are handled concurrently using goroutines handling incoming messages.
- Network Delivery Pipelines: Communication delivery exists off the main server logic. Dedicated goroutines (`WritePump`) receive messages through buffered channels (`SendChan`) and send them asynchronously, allowing the server to continue flowing without freezing on slow network operations.

Serial Parts:

- Dinic's algorithm: The highest level flow calculation for the map must wait for its root max-flow calculation to resolve linearly before bisecting out the two child threads recursively.
- Routing Inferences: Each specific A* Overlay query from source exactly to a target resolves serially as queue popping dictates strict deterministic chronological discovery, ensuring optimal guaranteed shortest path properties.
- Traffic Record Accruals: Applying weighted increments, resolving lock matrices safely across individual nodes, enforces strict single-thread gating per specific edge updating, preventing data races by using strict RWMutexes on internal maps.
- Parsing & CSR build: Parsing the raw OSM files, building the Nodes[] and Edges[] list, and building the Offsets array is done sequentially to preserve ordering and correct map build.
- Path reconstructions: At the end of the routing algorithm we traverse the pointers back and reconstruct the path sequentially.
- Sorting: All sorting (in Inertial Flow and sorting elements by cells for caching) is done using standard sequential sorting algorithms, since we don't have enough threads to benefit.
- The client side: All operations on the client side are done sequentially since javascript is single-threaded.

Concurrency Tools

The system uses many synchronization tools to handle goroutines:

- Locks - Sensitive actions are guarded by a mutex: The traffic store holds a mutex to guard concurrent array edits and snapshots are done safely with no partial writes or broken arrays. Every session and every simulated car holds a mutex to ensure that only one update can modify their internal state at a time. The critical sections were shrunk as much as possible.
- Atomics - Reading and writing the edge's weights is done atomically. There is no lock in reading or updating weights, which is essential since weights are accessed extensively in every route request, ping handling and background worker.
- Channels: Channels are primarily used to pass tasks between workers. For example, each session has a SendChan channel where goroutines can queue their messages without blocking, and a WritePump worker handles delivery to the client. Channels are also used to implement worker pools: tasks are inserted into a "jobs" channel that goroutines read from and execute. WaitGroups act as barriers, waiting for all workers to finish before processing.

Benchmarks

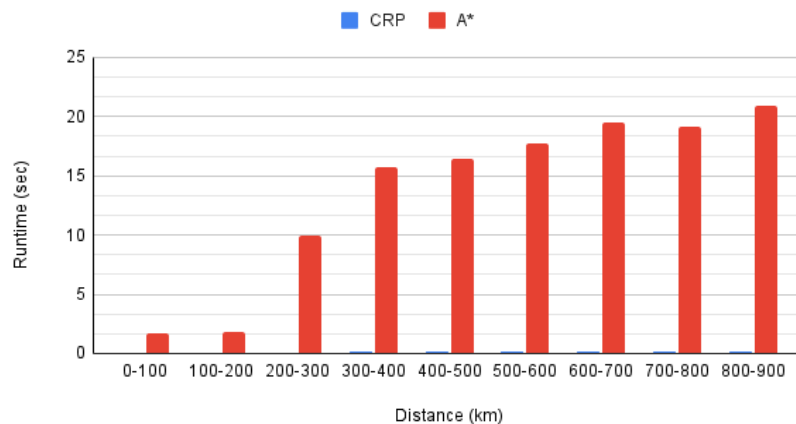
We set up benchmarks to show our algorithm's scalability, our server performance, and compare our implementation of customizable route planning against standard A* and dijkstra executions. The tests have been done on a processor: 11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz 3.00 GHz, 4 Cores, 8 Logical Processors.

CRP vs A*

In this part we calculated the shortest route between two randomly generated nodes using A* and Customizable Route Planning, increasing the distance between the nodes every time. The results are shown in the following table (the runtime is measured in ms):

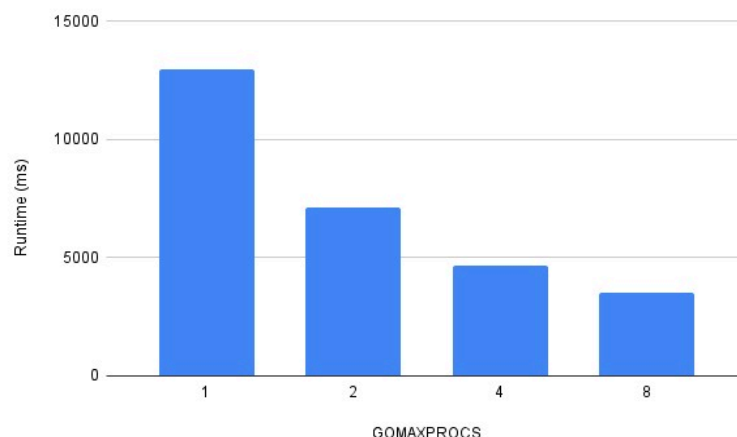
Distance (km)	0-100	100-200	200-300	300-400	400-500	500-600	600-700	700-800	800-900
CRP	92	73	105	169	185	191	187	222	219
A*	1722	1797	9926	15705	16420	17782	19508	19192	20876
Speedup	x19	x25	x94	x93	x89	x93	x104	x86	x96

For 800-900km, our algorithm found the route in ~220ms while A* took ~21s. The gap is illustrated perfectly when we convert this table into a chart, the blue towers is CRP's runtime:



Customization Scalability

In this part we tested the runtime of the customization phase as GOMAXPROCS increases:

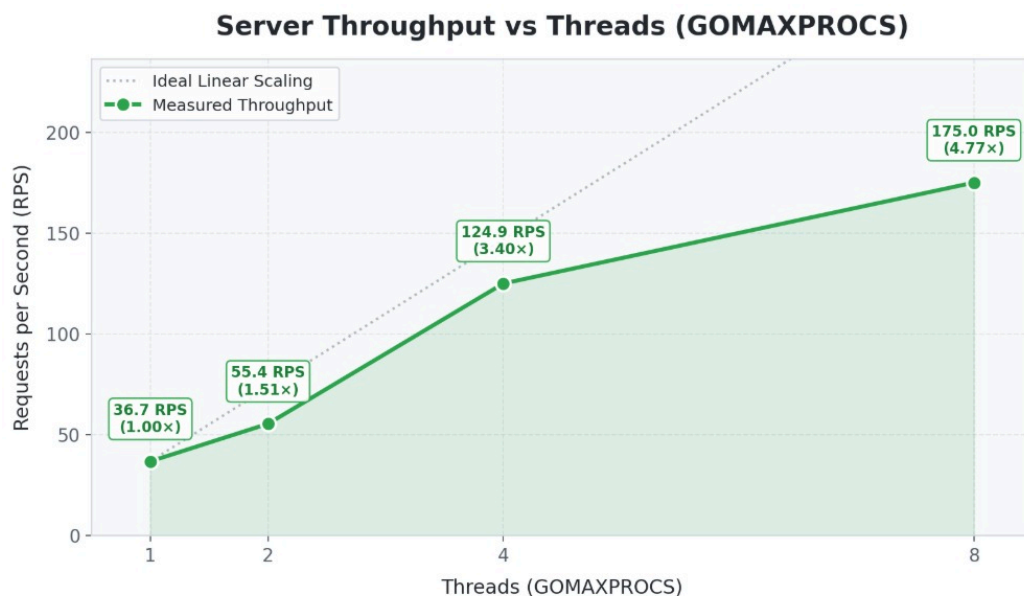


Server Scalability

The system was flooded with 15,000 randomized routes-calculation requests across the entire map. The load test maintained 50 active concurrent connections (simulating a heavy traffic spike of route requests). This workload was rerun while artificially restricting the Go runtime's core thread limit (*GOMAXPROCS*) to 1, 2, 4, and 8 threads to measure true scaling.

Results & Throughput Scaling

The results confirmed strong throughput scaling:



1 Thread (*GOMAXPROCS*=1): The engine maxed out at ~36 Requests Per Second (RPS).

8 Threads (*GOMAXPROCS*=8): The throughput jumped to ~175 Requests Per Second.

Latency Percentiles Analysis

The experiment evaluated the latency using Percentiles. Imagine lining up all 15,000 route requests from the fastest calculated path to the slowest:

P50 (Median): The exact middle request (the 7,500th path). 50% of the users received their routes faster than this time. This represents the standard, "typical" user experience.

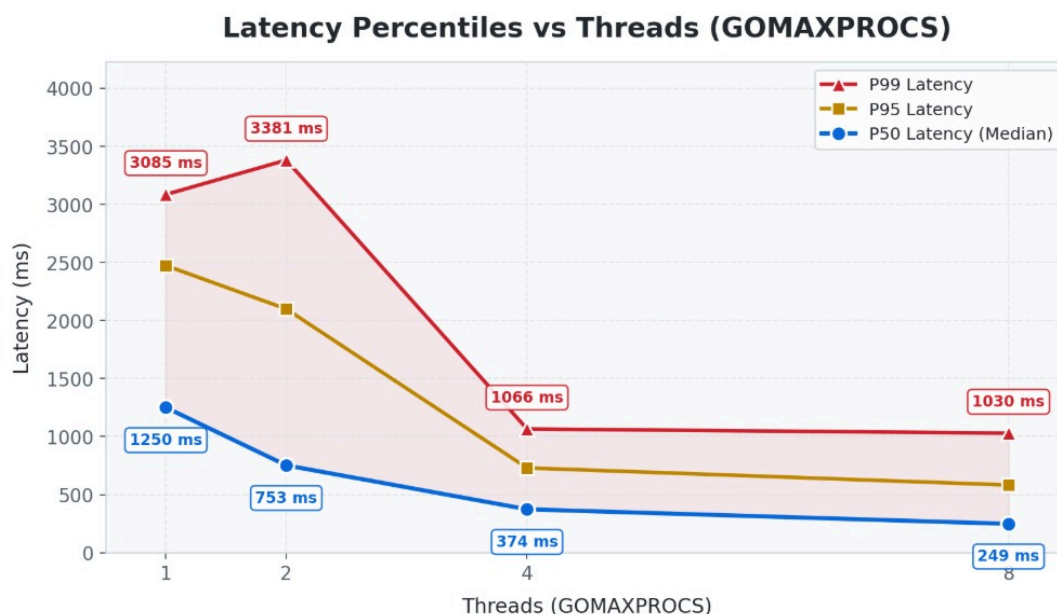
P95 (Tail Latency): The request near the back of the line, 95% of routes were faster than this

P99 (Extreme Tail): The absolute worst-case scenario, 99% of routes were faster

Results & The Concurrency Tradeoff

On 1 Thread, the lack of parallel workers created a massive traffic jam of requests. The median query (P50) processed in ~1.2 seconds, the P95 lingered at ~2.5 seconds, and the absolute worst-case P99 spiked over 3 seconds.

Upon releasing 8 Threads, the system instantly distributed the graph-traversal load. The P95 was crushed from 2.5 seconds down to 550ms, and the P99 crushed to just 1 second.



Although those amazing results - there is only one strange thing - why is the P99 with 1 thread lower than with 2 threads?

This illustrates the classic tradeoff we saw in our course: Concurrency is not free (because of the costs we saw in class of threads synchronization and false sharing).

So we can see that only for the smaller amount of calculations, like in the P50, we might benefit from lowering the thread count, like we saw the 1 thread vs. 2 threads, but in the long run - especially in large and global systems that needs to make enormous amounts of complex calculations - the benefit of the parallelism squeezes the overhead, **if we use the threads right, safely and synchronized**.

The Power Of Goroutines

The server's performance results demonstrate its ability to handle many concurrent tasks. At every given moment, the server handles many concurrent tasks: processing incoming requests, computing shortest routes, tracking real time vehicle positions, updating live edge weights, and considering optional reroutes. By dividing the work into tasks and coordinating goroutines, the server handles large workloads with low overhead and high throughput.